面向下一代硬件的性能调优

作者：袁江 王浩鹏

时间：2026.3

课程背景与目标

课程背景:

随着昇腾AI处理器架构的持续演进，下一代AI处理器架构在硬件设计针对用户性能调优的痛点设计了新的调优能力，同时下一代AI处理器也引入了一些新的业务功能以及相应的调优能力。

课程目标:

- 熟悉下一代芯片的整网调优能力
- 熟悉下一代芯片的算子上板调优能力

前置知识:

- 《基于下一代Ascend平台的Ascend C算子编程概述》
  https://www.bilibili.com/video/BV12fAdzcEDe/?spm_id_from=333.1387.collection.video_card.click&vd_source=40a01afa50d31f7f76277e5baa2faa1f
- 《基于下一代Ascend平台的SIMT编程介绍》
  https://www.bilibili.com/video/BV1zNPwzaEeB/?spm_id_from=333.1387.homepage.video_card.click&vd_source=cdb72407c0b73451e8ecbc013b172a61

目录

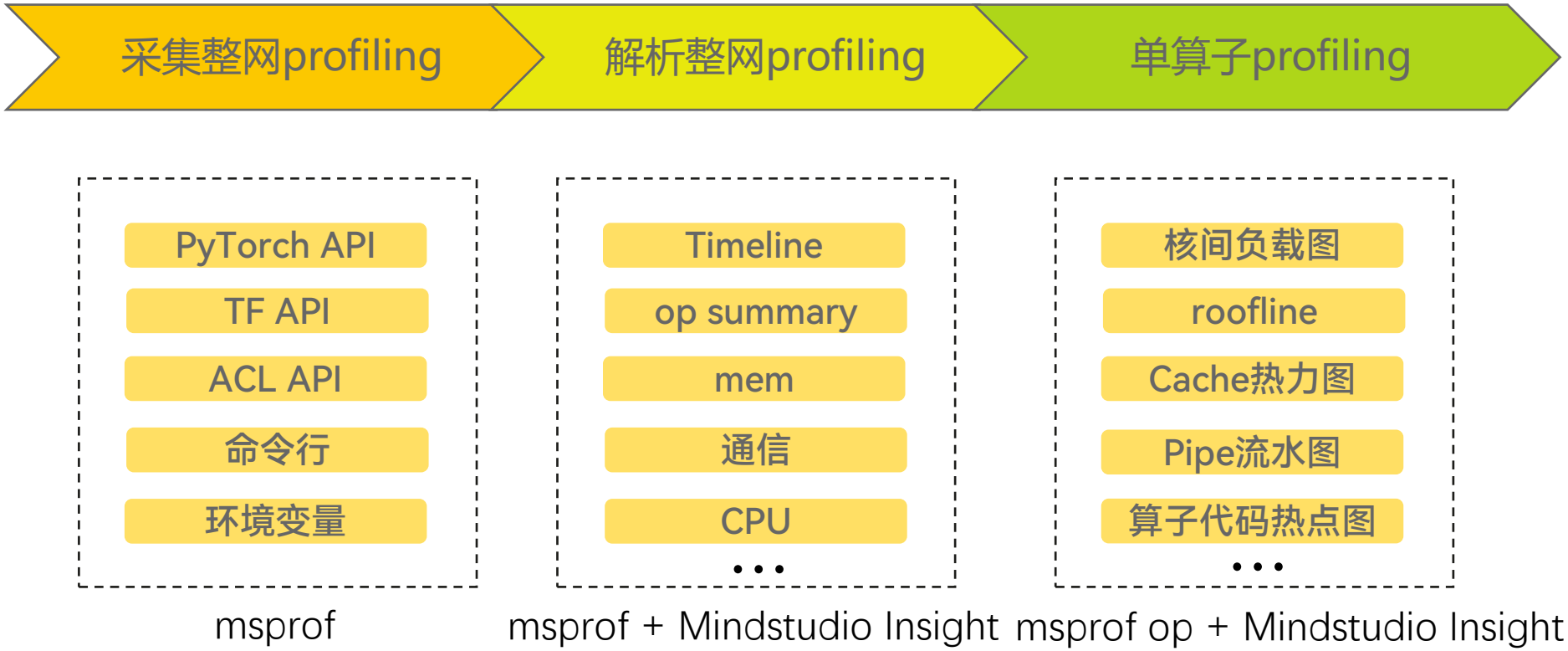
Part 1 性能调优整体流程

Part 2 下一代硬件整网profiling新特性

Part 3 下一代硬件算子profiling新特性

CANN

NPU性能调优流程


采集整网profiling
解析整网profiling
单算子profiling
PyTorch API
TF API
ACL API
命令行
环境变量
Timeline
op summary
mem
通信
CPU
...
核间负载图
roofline
Cache热力图
Pipe流水图
算子代码热点图
...
msprof
msprof + Mindstudio Insight
msprof op + Mindstudio Insight

整网profiling工具-msProf

采集

PyTorch

```
with torch_npu.profiler.profile() as prof:
        for step in range(steps):
                train_one_step(step, steps, train_loader, model, optimizer, criterion)
prof.export_chrome_trace('./chrome_trace_14.json')
```

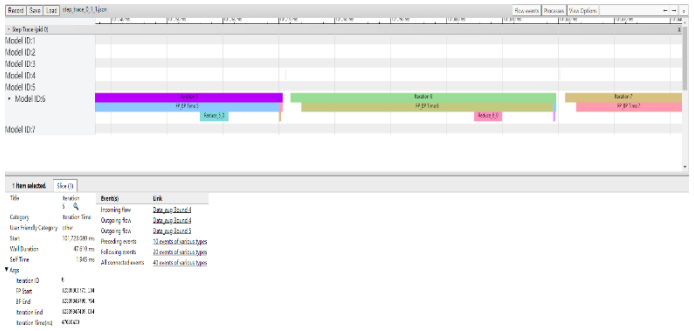
命令行

```
msprof --application="/home/projects/MyApp/out/main" --output=/home/projects/output
```

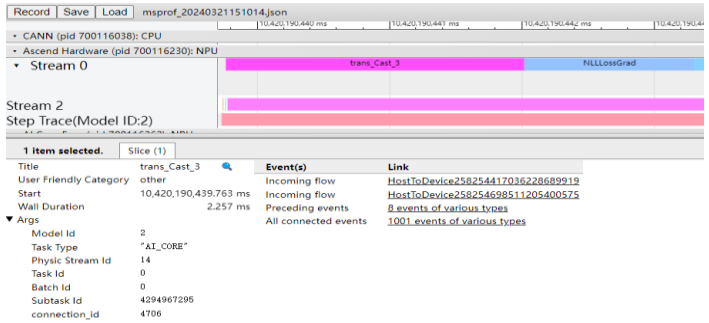
解析

```
msprof --export=on --output=<dir> [--type=<type>] [--iteration-id=<number>] [--model-id=<number>]
```

迭代耗时



算子耗时



算子统计

Device_id	Model Name	OP Type	Core Type	Count	Total Time(us)	Min Time(us)	Avg Time(us)	Max Time(us)	Ratio(%)
1	ge_default_20230919170146_121	Conv2D	AI_CORE	53	6220.324	57.091	117.365	789.496	20.158
1	ge_default_20230919170146_121	Conv2DBa	AI_CORE	52	5395.378	14.76	103.757	661.873	17.484
1	ge_default_20230919170146_121	GetNext	AI_CPU	2	4304.476	1991.79	2152.238	2312.686	13.949
1	ge_default_20230919170146_121	BNTraining	AI_CORE	53	2920.348	17.69	55.101	183.284	9.464

可视化工具-MindStudio Insight

简介

针对集群场景待分析的数据量大，数据分布于不同的机器上等特点， MindStudio Insight提供适配昇腾平台的框架Profiling可视化呈现能力，构建框架+CANN软件栈+昇腾硬件端到端的调优能力，提供多维度的性能分析视角，包括计算、通信、内存等多样化的数据呈现，使能开发者在大模型集群调优场景下能快速发现模型性能瓶颈，持续提升模型性能。

可视化调优工具

收集处理器硬件、操作系统、进程/线程、函数等各层次的性能数据，分析系统性能指标，定位到系统瓶颈点及热点函数，给出优化建议，辅助用户快速定位和处理软件性能问题。

性能分析

计算性能分析	通信性能分析	内存性能分析
框架算子下发耗时	慢卡分析	应用内存使用
算子调用栈显示	通信域划分	算子内存申请
执行关联	通信矩阵展示	算子内存使用

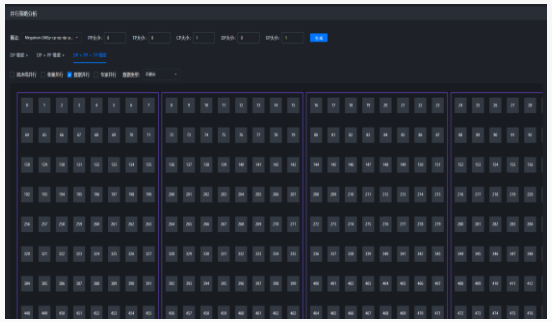
性能数据可视化

性能问题识别

MindStudio Insight提供timeline流水图、通信分析、内存分析等多种形式帮助用户快速发现异常点，识别性能瓶颈

并行策略分析及调优技术

性能问题分析



基于拓扑和并行策略，结合可视分析链路进行调度分析，定位通信慢、计算慢等性能问题根因

算子profiling工具-msOpProf

采集

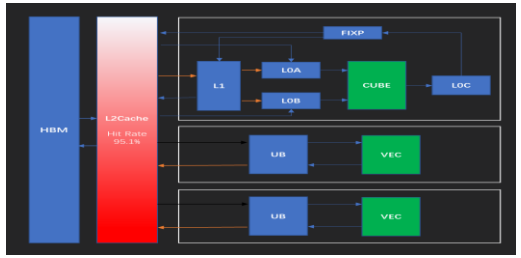
```
msprof op --output=$HOME/projects/output $HOME/projects/MyApp/out/main

msprof op simulator --soc-version=Ascendxxxyy --output=./output_data ./add_custom_npu
```

解析

核间负载图

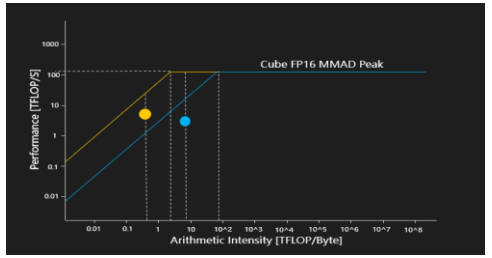
直观展示算子运行时，每个核的负载均衡情况

计算内存热力图



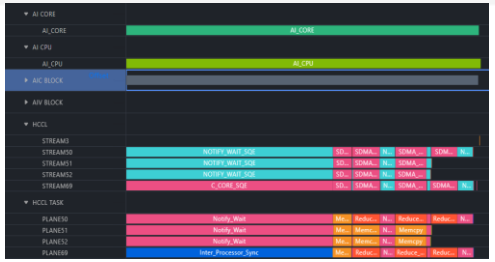
资源维度展示利用率，全局视角识别瓶颈

Roofline性能分析



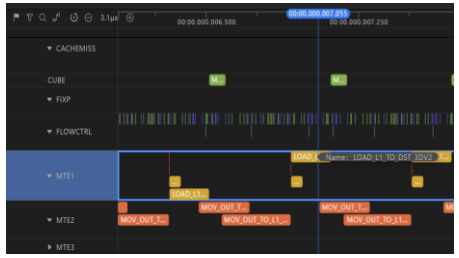
理论维度展示优化空间，快速评估瓶颈优化收益

通算流水分析



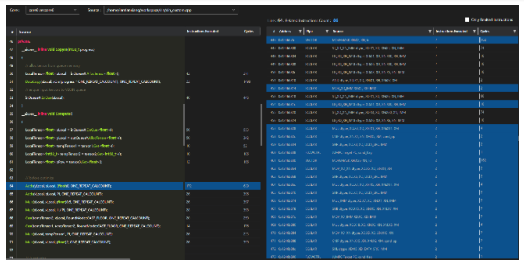
提供通算融合算子的流水图、性能数据展示

指令流水图



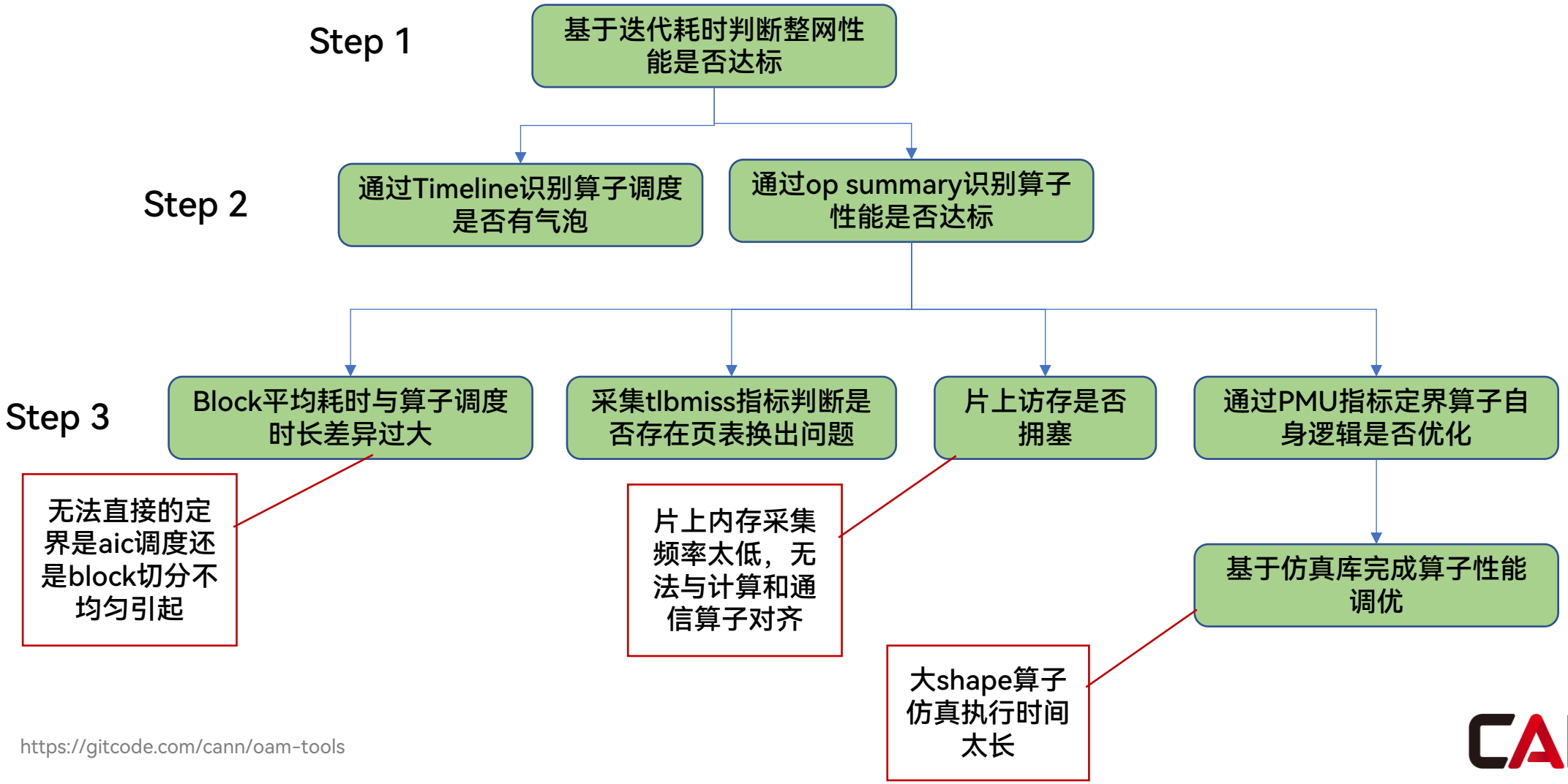
指令维度展示时序关系，关联调用栈快速追踪瓶颈位置

代码性能度量



代码维度展示片段耗时，分析热点函数优化可行性

NPU性能调优场景和最佳实践



https://gitcode.com/cann/oam-tools

CANN

本章小结

- **介绍性能调优整体流程**
  - ✓ 现有的调优工具和基础功能
  - ✓ 现有调优流程的痛点

目录

Part 1 性能调优整体流程

Part 2 下一代硬件整网profiling新特性

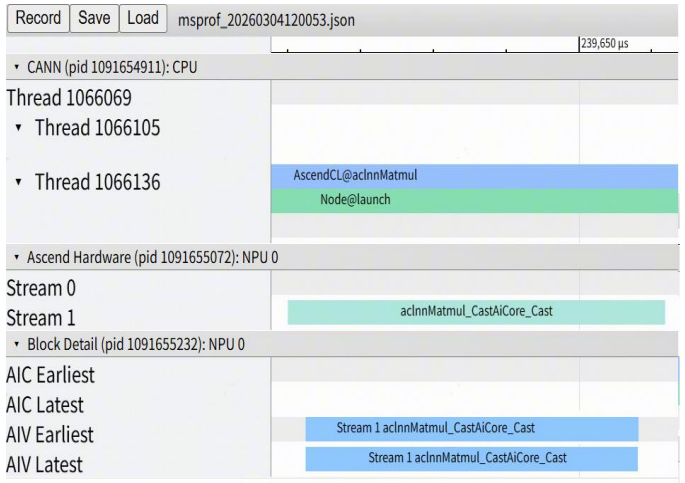
Part 3 下一代硬件算子profiling新特性

AICore算子首尾block profiling-算子调度&切分问题定界

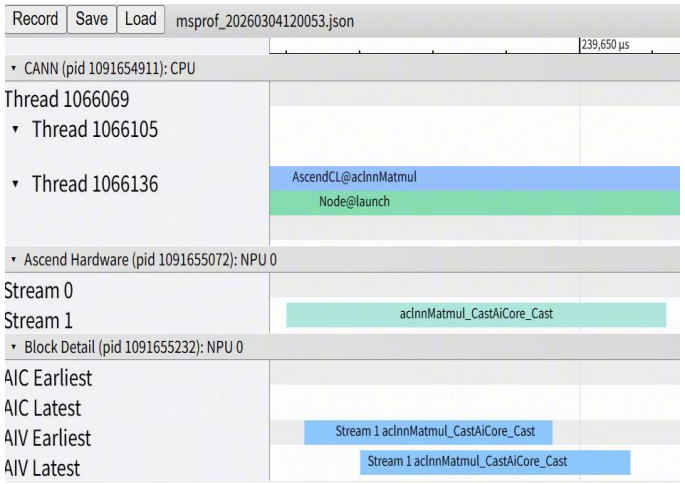
使能

msprof –application="/home/projects/MyApp/out/main" --output=/home/projects/output --task-block=on

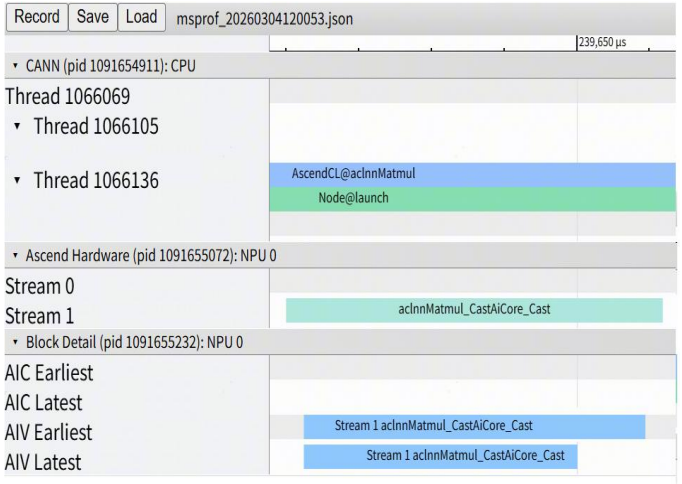
数据



正常的aiv算子数据



AICore调度问题



算子block切分不均匀

支持开发者在整网调度中观察到第一个和最后一个调度的block性能数据，支持定界调度和算子的性能问题

https://gitcode.com/cann/oam-tools

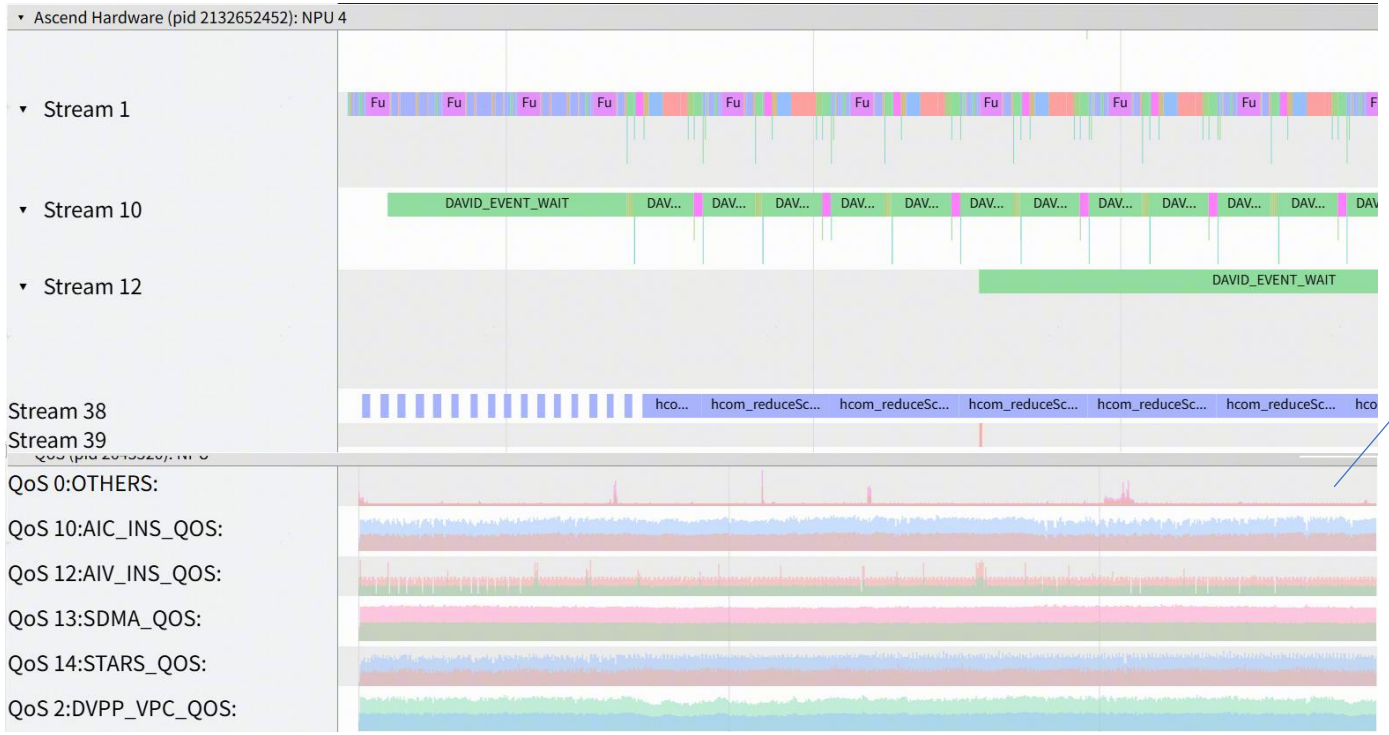
CANN

片上内存带宽统计-算子访存效率和抢占

使能

msprof –application=“/home/projects/MyApp/out/main” --output=/home/projects/output ---sys-hardware-mem=on

数据



默认场景下，没有QoS控制，profiling只会呈现QoS 0这一行
用户如果配置了加速器级别的QOS控制时，可以统计到不同加速器级别的带宽

采集频率从100hz提升到10000hz，开发者可以看到不同任务执行期间的带宽变化

CCU profiling – CCU内部关键性能打点

CCU是下一代硬件新增的集合通信专用的计算单元，可以有效降低集合通信对片上内存的访问压力，profiling增加了采集CCU性能数据的能力

Timeline展示CCU内部的关键指令执行耗时

统计不同CCU任务各个channel的带宽

Device_ic	Channel Id	Timestamp(us)	Max Bandwidth(MB/s)	Min Bandwidth(MB/s)	Avg Bandwidth(MB/s)
0	0	1766558554812626.667	10803	5923	7134
0	1	1766558554812626.667	11909	6161	7013
0	2	1766558554812626.668	11791	6025	7211
0	3	1766558554812626.668	9901	6045	6832
0	4	1766558554812626.669	10602	5878	6678
0	5	1766558554812626.670	11959	6123	6903
0	6	1766558554812626.670	10521	6134	7103
0	7	1766558554812626.671	11853	6078	6843
0	8	1766558554812626.672	11531	5967	7212
0	9	1766558554812626.672	11732	5896	7321
0	10	1766558554812626.674	10943	6071	6843
0	11	1766558554812626.674	11326	6069	6934
0	12	1766558554812626.691	10831	6071	6912
0	13	1766558554812626.691	11943	5832	7045
0	14	1766558554812626.691	11762	5799	6832
0	15	1766558554812626.691	10723	6109	6789
0	16	1766558554812626.692	11901	5938	7041

https://gitcode.com/cann/oam-tools

CANN

多进程profiling – 推理场景多进程并发调优

多个进程在一个NPU执行任务时，可以同时采集业务级profiling数据

本章小结

- 下一代硬件整网profiling新特性
- ✓ AICore算子首尾block profiling
- ✓ 片上内存带宽
- ✓ CCU profiling
- ✓ 多进程profiling

目录

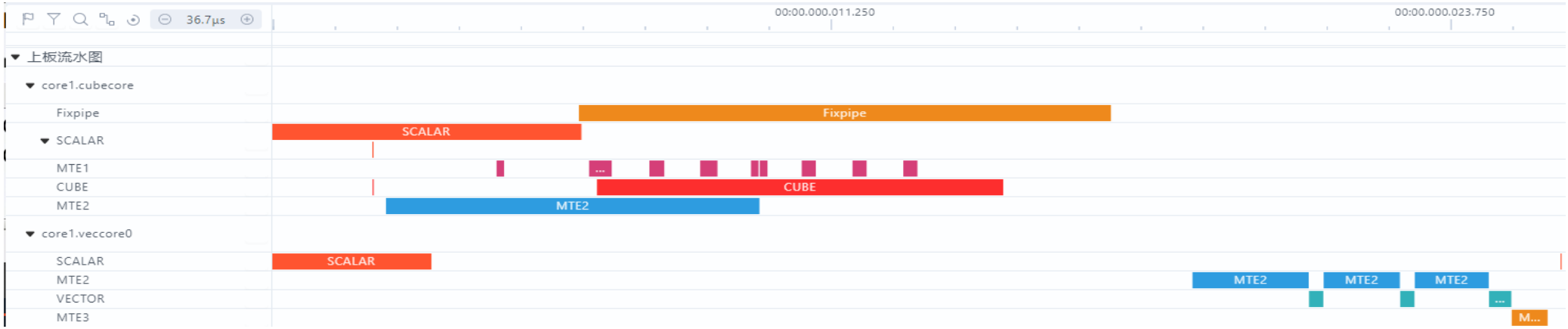
Part 1 性能调优整体流程

Part 2 下一代硬件整网profiling新特性

Part 3 下一代硬件算子profiling新特性

Pipe流水图-快速识别算子内部计算和内存搬运流水bubble

场景：用于快速识别算子内部计算和内存搬运流水bubble。
价值：将性能分析时间从原来的天/小时级缩短到分钟/秒级，同时增强数据准确性。
痛点：Pipe流水图很难关联到代码。



解决方法：通过AscendC::MarkStampImpl在算子代码中插入自定义标志点，用户可以根据流水图呈现结果关联到代码

```
template <typename T>
__aicore__ inline void SimtComputeShell(GM_ADDR x
{
    __gm__ T* dst = (__gm__ T*)x;
    __gm__ T* src0 = (__gm__ T*)y;
    __gm__ T* src1 = (__gm__ T*)z;
    // VF_CALL启动SIMT VF子任务
        AscendC::MarkStampImpl<PIPE_V,14>();
    AscendC::Simt::VF_CALL<SimtCompute<T>>(AscendC
        AscendC::MarkStampImpl<PIPE_V,15>();
}
```

ID为14的打点　　ID为15的打点

PC Sampling-定位SIMT算子阻塞代码和指令

场景：SIMT算子代码复杂，有较多分支，需要找到阻塞代码和指令
应用：采集硬件上报的指令阻塞状态并分析得到Stall Sampling表，让用户一眼找出阻塞比例最大的指令和状态

Tips：
支持悬浮展示九种Stall状态详情
详细展示各个阻塞状态采样次数
采样次数 ≠ 实际执行次数

#	Source	Inst...	Stall Sampling(All Sa...	Stall Sampling(Not I...
2	* Copyright (C) 2026. Huawei Technologies Co., Ltd. All rights reserved.			
3	*/			
4	#include "kernel_operator.h"			
5	using namespace AscendC;			
6				
7	template <typename T>			
8	__simt_vf__ __aicore__ LAUNCH_BOUND(2048) inline void SimtCompute(__gm__ T *dst, __gm__ T *src0, __gm_			
9	{	8	0.00%	0.00%
10	int idx = Simt::GetThreadIdx<0>();	48	0.00%	0.00%
11	int blockIdx = Simt::GetBlockIdx();			
12	int tmp = 10;			
13	for (int i = idx; i < idx + 2000; ++i) {	161...	0.01%	0.01%
14	auto val1 = src1[i];	161...	1.79%	1.70%
15	auto val2 = src0[i];	161...	21.47%	21.46%
16	auto val3 = val1 * val2 + tmp;	8093	53.81%	53.98%
17	if (val3 % 4 == 0) {	323...	0.02%	0.02%
18	val3 = val2 * tmp + val1;			
19	}			
20	dst[i] = val3;	242...	22.06%	21.99%
21	}			
22	}	15	0.00%	0.00%
23				
24	template <typename T>			
25	__aicore__ inline void SimtComputeShell(GM_ADDR x, GM_ADDR y, GM_ADDR z)			

指令关联代码

#	Address	Pipe	Source	Instruction...	GPR Count	GPR Status	Stall Sampling(All Sa...	Stall Sampling(Not Is...
2	0x1200443f70bc	VECTOR	VF_SIMT Xn, Xm, ...	4	0		0.00%	0.00%
3	0x1200443f70c4	SCALAR	MOVI	4	0		0.00%	0.00%
4	0x1200443f70c8	SCALAR	DCCI	4	0		0.00%	0.00%
5	0x1200443f70cc	SCALAR	END	4	0		0.00%	0.00%
6	0x1200443f70d0	VECTOR	S2R	8	2		0.00%	0.00%
7	0x1200443f70d8	VECTOR	S2R	8	3		0.00%	0.00%
8	0x1200443f70e0	VECTOR	SHFI	8	3		0.00%	0.00%
9	0x1200443f70e8	VECTOR	IADD.i	8	4		0.00%	0.00%
10	0x1200443f70f8	VECTOR	IADD.i	8	5		0.00%	0.00%
11	0x1200443f7108	VECTOR	START_DVG	8	3		0.00%	0.00%
12	0x1200443f7110	VECTOR	MOVI	8	4		0.00%	0.00%
13	0x1200443f7118	VECTOR	FMUL.i	8093	6		0.01%	0.01%
14	0x1200443f7128	VECTOR	FMUL.i	8093	8		0.01%	0.01%
15	0x1200443f7138	VECTOR	FMUL.i	8093	10		0.00%	0.00%
16	0x1200443f7148	VECTOR	LDG	8093	11		1.78%	1.70%
17	0x1200443f7150	VECTOR	LDG	8093	10		21.46%	21.46%
18	0x1200443f7158	VECTOR	FMUL.i	8093	9		53.81%	53.98%
19	0x1200443f7168	VECTOR	FMUL.i	8093	10		0.00%	0.00%
20	0x1200443f7178	VECTOR	Lop3	8093	9		0.00%	0.00%
21	0x1200443f7188	VECTOR	ISETP.i	8093	7		0.01%	0.01%
22	0x1200443f7198	VECTOR	SEL	8093	7		0.01%	0.01%
23	0x1200443f71a0	VECTOR	ISETP	8093	5		0.01%	0.01%
24	0x1200443f71a8	VECTOR	IADD.i	8093	4		0.01%	0.00%
25	0x1200443f71b8	VECTOR	STG	8093	3		0.81%	0.75%

Total: 73751
Active: 181
IBuf_Empty: 663
Nop_Cycles: 189
Others: 63
Register_bank_conflict: 84
Resource_conflict: 151
Scoreboard_Not_Ready: 72420

53.81% 53.98%

CANN

SIMT 寄存器分析-通过细粒度寄存器状态分析占用逻辑的合理性

#	Address	Pipe	Source	Instruction...	GPR Count	GPR Status	Stall Sampling(All Sa...	Stall Sampling(Not Is...
18	0x1200443f7158	VECTOR	FMUL.i	8093	9		53.81%	53.98%
19	0x1200443f7168	VECTOR	FMUL.i	8093	10		0.00%	0.00%
20	0x1200443f7178	VECTOR	Lop3	8093	9		0.00%	0.00%
21	0x1200443f7188	VECTOR	ISETP.i	8093	7		0.01%	0.01%
22	0x1200443f7198	VECTOR	SEL	8093	7	写入	0.01%	0.01%
23	0x1200443f71a0	VECTOR	ISETP	8093	5		0.01%	0.01%
24	0x1200443f71a8	VECTOR	IADD.i	8093	4	先读后写	0.01%	0.00%
25	0x1200443f71b8	VECTOR	STG	8093	3	读出	0.81%	0.75%

场景：需要针对SIMT算子的寄存器读/写使用情况进行细粒度分析。

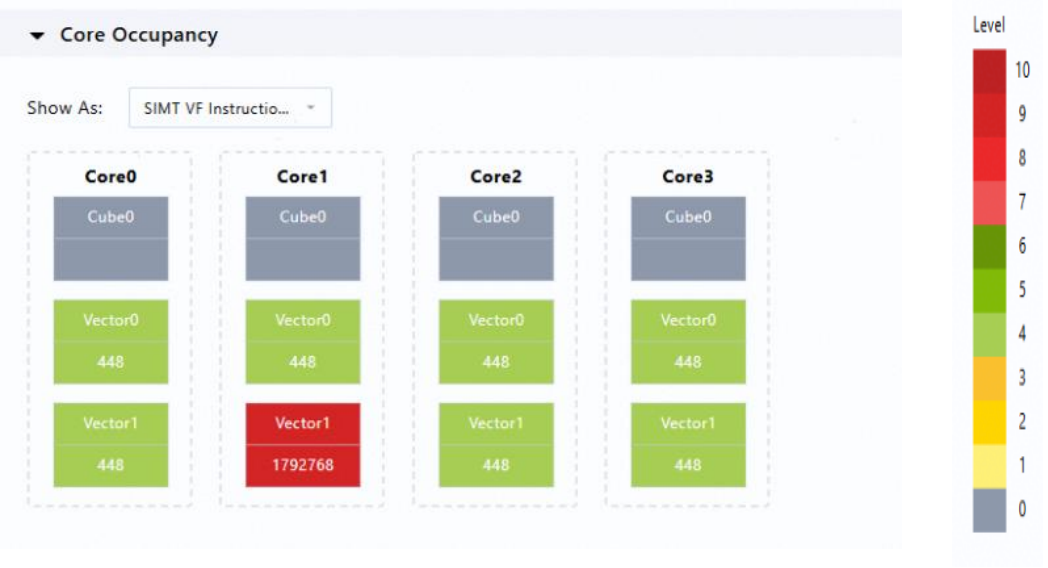
应用：在SIMT算子运行过程中记录指令运行时的寄存器占用个数以及各寄存器状态

1. GPR Count：展示该指令执行时寄存器被占用的数量

2. GPR Status：展示各寄存器读写状态，用于分析寄存器占用逻辑是否合理。

CANN

SIMT 指令发射效率统计-核间负载分析手段



场景：需要针对各核间的负载是否均衡进行分析

应用：
Core Occupancy：核间负载图上显示指令发射次数，用颜色表示均衡程度，这里core1 vector1 的指令执行次数明显过高。
IPC(Instruction Per Cycle)：展示每个核的指令发射效率，可以看出core1 vector1 效率较差。

Tips：建议从切分策略等角度出发去调优

Instruction Per Cycle

	Core0	Core1
vector0	2.0550458431243896	2.0550458431243896
vector1	2.0550458431243896	1.4891595840454102

CANN

本章小结

- 下一代硬件msOpProf新特性
- ✓ Pipe流水图
- ✓ PC Sampling
- ✓ SIMT 寄存器分析
- ✓ SIMT 指令发射效率统计

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN